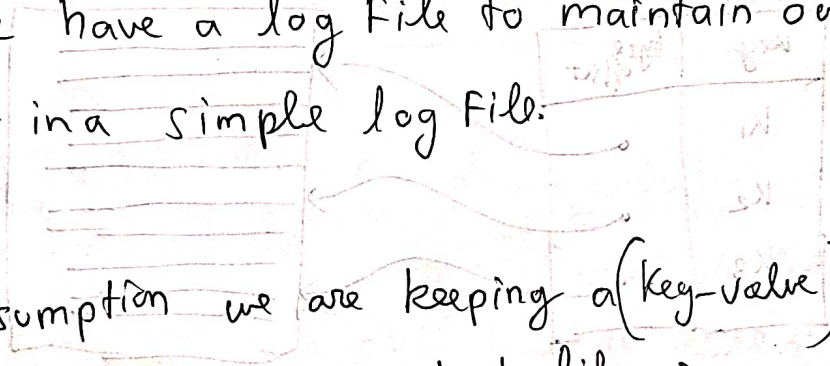
(Designing Data Intensive Applications)

(Chapter-3 Storage And Retrival)

Starting from Scratch. (Lets say we don't have any database).

Lets say we have a log File to maintain our entries just in a simple log File.



LogFile → Assumption we are keeping a (key-value) pair and store it in a txt file in append mode

For any new write, we just append only and we do not modify any previous content.

Q) How will you read data for a particular key?

A) Basically scan entire file and take O(N) operation.

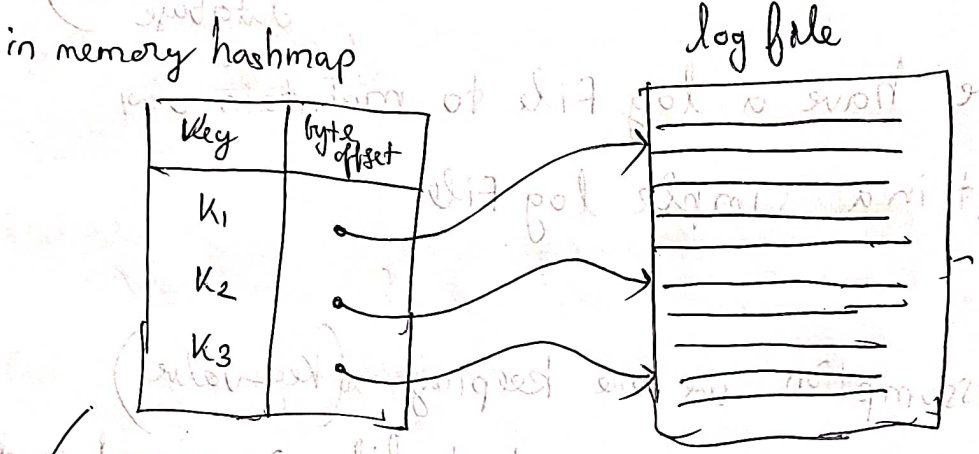
To optimize it or improvise over it, then we can maintain indexes. [We already have a slide on indexes please visit that].

Q) How will you index data on disk?

Indexing strategy

↳ Create in-memory hashmap

↳ Every key is mapped to byte offset in file.



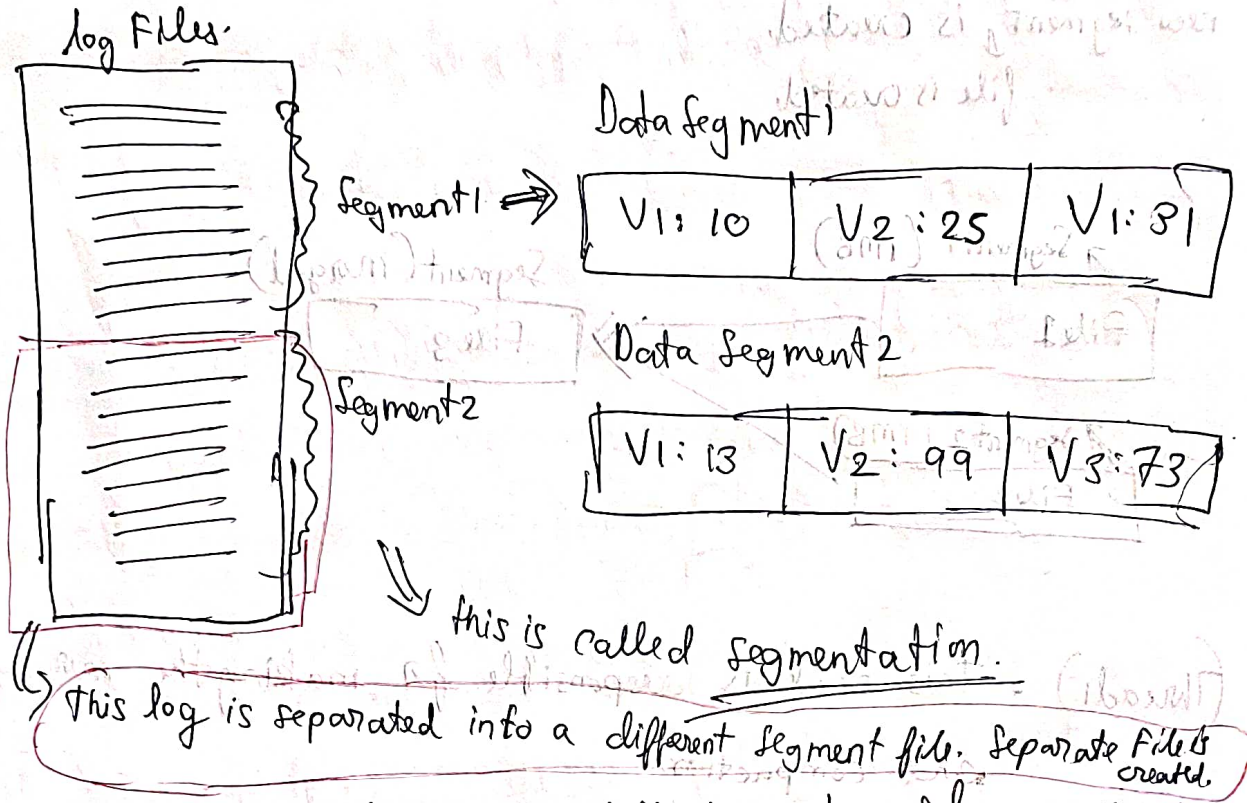
↳ This storage pattern is used by (Bit Cask)

Can be used for small storage like where counter can be stored

VideoID	NoOfPlays

Q) How do we ensure that memory does not run out?

Idea is to break the log files into segments.

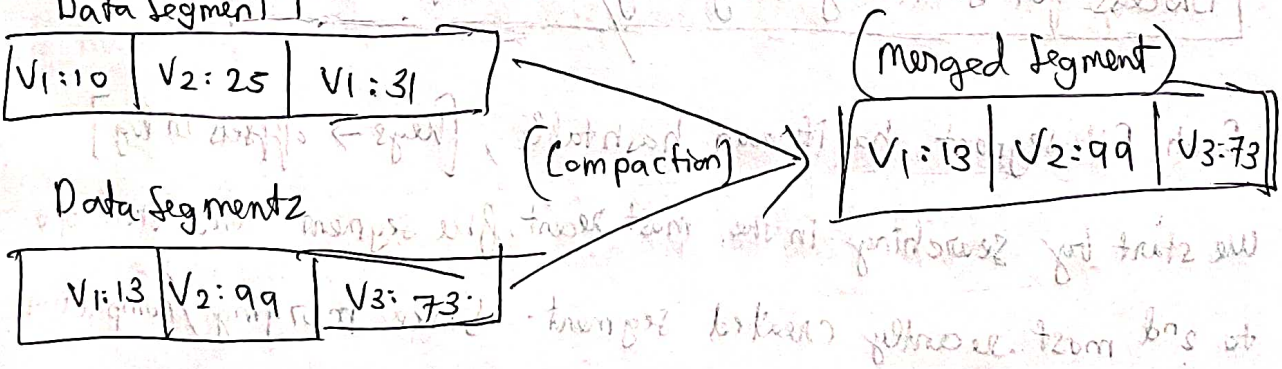


Now segmentation is fine, But still how to release memory

Compaction

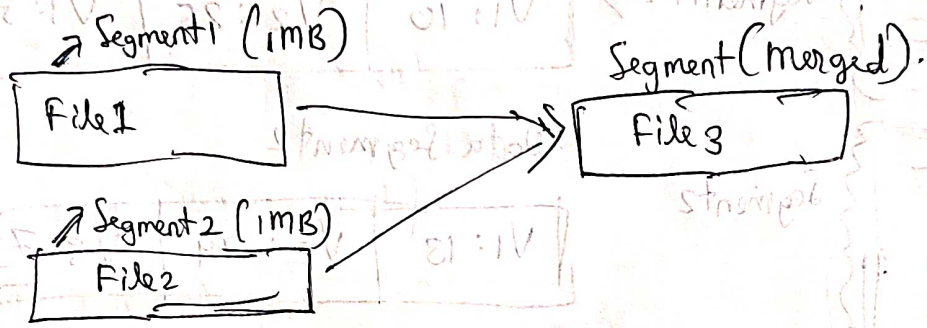
We take two Data Segments, Segment 1 and Segment 2.

We merge them and only keep the most recent updated value in the ~~segment~~ new Segment



Segment files are never modified. When two segments are merged, then at that point a new file is opened and a new segment is created.

file is created



(Thread1) = This read is responsible for making the merging and compaction.

(Thread2) = It is responsible for reading requests and write requests

(xxxx)

After the merging process is complete, we route the read requests to the new segment and delete the old bigger segment.

Process for Searching any Key / Lookup Process

Each File segment has its own hashtable, [Keys → offsets in log]

We start by searching in the most recent file segment and then go to 2nd most recently created segment. Since merging/compaction reduces the number of segments. Hence no. of Hashmap lookups is less.

Real Life Implementation of this Architecture.

Two Requirements-

1. Keep in-memory hash tables for each Segment File
2. Keep the segment files on the disk.

File Format: Maintaining our segments and files so that our disk takes minimum space.

↳ Storing the file in binary format, [.bin]

Record Deletion: In Cassandra, how we do it, is we maintain a (tombstone) architecture, we just mark the entry as (special) deleted and don't do anything.

During Read requests we don't read that entry.

And during compaction, automatically it gets deleted.

[Crash Recovery]: Since we are maintaining the hashmaps of (key → offset) in-memory. In case of server crash, if hash-map is lost, we will have to reconstruct it again.

[Bitcask] → a software company, does this recovery by storing a snapshot of hash-map in the disk as well.

[Partially written Records]

Lets say database crashes halfway through when you are appending a record to the log.

Basically wrong data is stored Now in the log.

(User intended) [name: Wrishaw Bhattacharyya].

But (mid way crash)

[name: Wrishaw Bhat]

How to avoid this?

Bitcask uses checksums to avoid this ~~kind~~ kind of situation.

1) Checksum Calculation: when data is written to a log or a file, a checksum is computed from the original saree.

During write, it uses the (entry, timestamp) everything.

So [name: Wrishaw Bhattacharyya] timestamp ⟹ Checksum1.

During Read

[name: Wrishaw Bhat] writeTimestamp ⇒ Checksum2 is calculated again.

[checksum1 != checksum2] ⟹ (Corrupt Data)

(Concurrency Control).

For write operations, if we have multiple writer threads the issue can occur on who appends the last.

So we only give 1 writer thread, or introduce locks.

And multiple Reader threads.

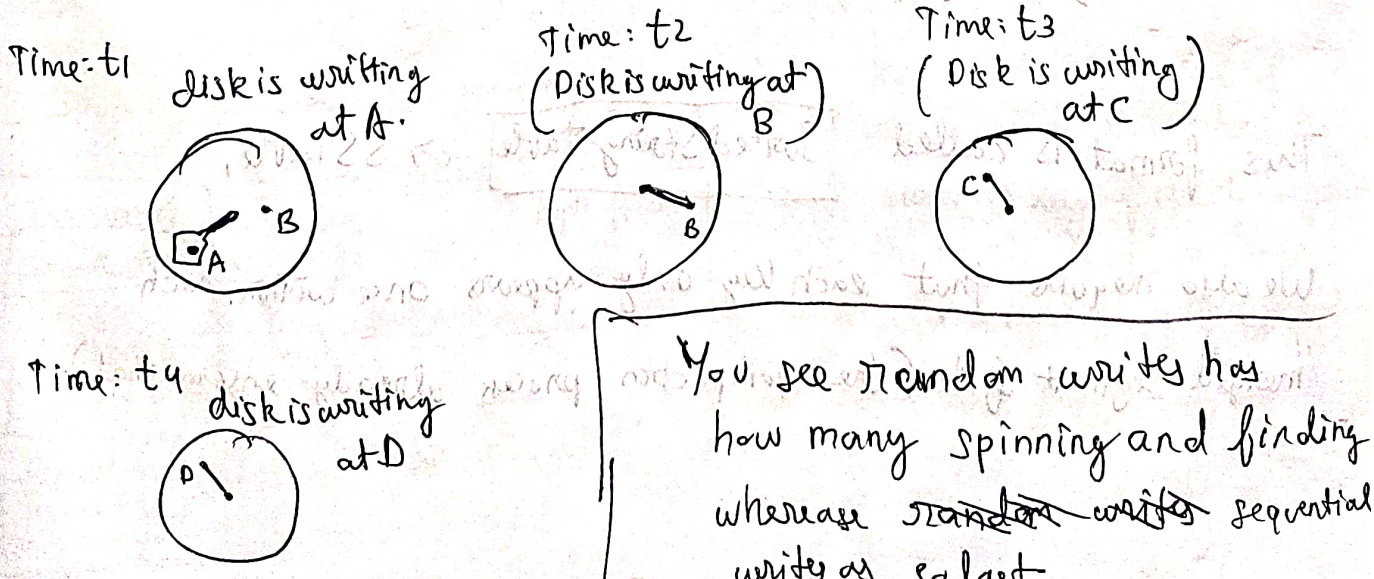
(☆☆☆☆)

(V.Imp Question)

Q) Why do we need append only log. Can't we just edit wherever it is required? Like scroll up and modify etc

A)

- Appending and segment merging is much faster as it is sequential writing. But if as above if we do a random write then Magnetic Spinning disk, Solid State drives (SSDs) do not perform so well.



You see random writes has how many spinning and finding whereas ~~random writes~~ sequential writes as so fast

Crash Recovery.

If System Crashes, if it was random writes, it might have happened, that some part of the edit is new, whereas some part is old.

In sequential write this is not the case. Corrupted write, we ~~saw~~ saw how to discard.

Limitations of this setup:

- Hash table must fit into the memory
- Range queries will take linear time complexity

SSTables and LSM-Trees.

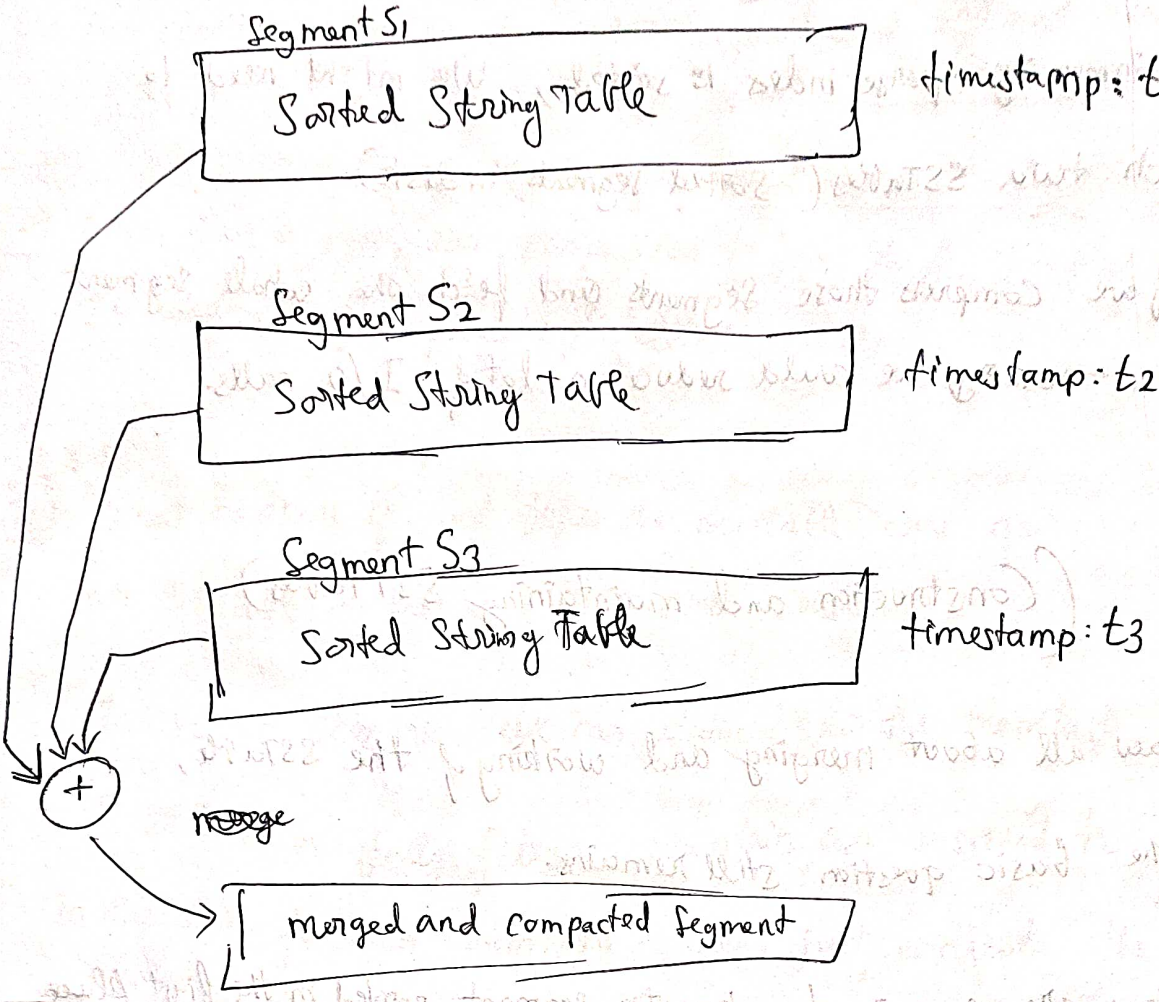
Let us assume here that the segment files are sorted by key. Each segment file is sorted by the key. We will see later how, it is acheived, but just consider it For now.

This format is called **Sorted String Table** or SSTable,

We also require that each key only appears once within each merged segment file (the compaction process already ensures that).

1. Merging of Segments:

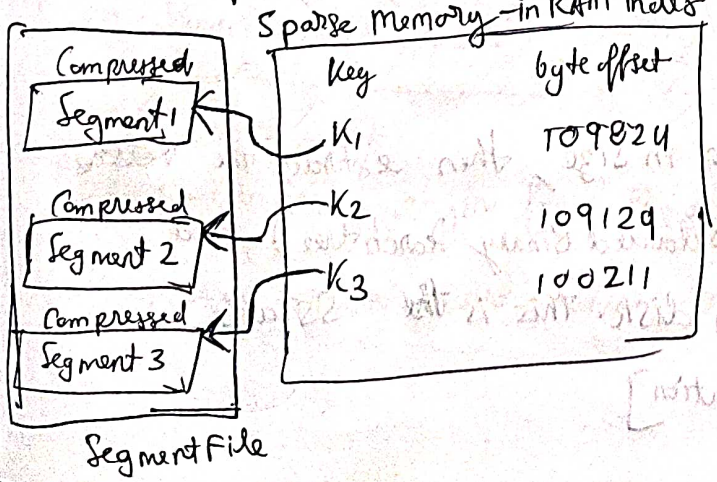
The Algo followed here is just like merge Sort.



If during merging, if we see that two same keys V_1: value1, V_1=value2, then we pick the key which is part of the SSTable, which was formed at a later timestamp.

(Searching for a key in a segment File)

2. Storing of indexes (in-memory): We don't keep all the keys of the segments, we keep the starting key of each segment so that using that one can find the key in the segment



⇒ Each segment file has its own Sparse in-memory index

(3) the segments can be kept in compressed block,

When the user searches (select * from where key='K1')

Our in memory sparse index is sorted, We might need to search two SSTables (sorted segments in disk).

So if we compress those segments and fetch the whole segment into in-memory, we could reduce a lot of I/O calls.

(Construction and maintaining SSTables)

We saw all about merging and working of the SSTable,

But the basic question still remains.

Q) How are we going to keep the segment sorted in the first place.

Any new segment that is been formed, needs to be sorted,

Any new entry shall be sorted order, How to do that?

A) (1) When a write comes, in add it to the balanced tree data structure (AVL tree, Red-Black tree). This is in memory and called mem-table.

(2) When mem table gets bigger in size, then extract the keys from the mem table (Balanced Binary Search tree) and populate into a file in disk. This is the SSTable.

[Sorted Segment Formation]

3) In order to serve a read request, first try to find the key in the memtable (which is in-memory getting constructed), then check on most recent on-disk segment and 2nd most recent segment.

4). From time to time, run compaction to keep no of SSTables small and take care of duplicate / deleted records.

(x x x)

Q) What happens if just before the memtable was going to create an SSTable on the disk, the server crashes.

A) In such a scenario, we can always for the memtable, which is still to be fully filled, maintain an unsorted commit log in the disk for each memtable. Then just recreate the mem table from the commit log.

(Making an LSM tree out of SSTables.)

This indexing structure of keeping a sparse index and and merging the SSTables as part of compaction, is called LSM tree. (Log-Structured merge tree). ⟹ Cassandra uses this.

→ (Similar to splunk)

Lucene → indexing engine for full text search uses similar method.

(key, value) (key = word), value = (document ID list)

Performance optimisations on LSM Trees.

* Read can be slow. As for searching for any key.
we check in the mem table, then we go on searching in the most recent SSTable created {utilize its sparse-in-memory-index}. the go to the next most recent and so on.
So we use Bloom filter on each segment which gives us approximately if the key is present in the segment or not.

* Deciding on the timing of Compaction & merging:
We need to fix the order and timing in which we should ask for apply Compaction, so user is not affected.

Size Tiered Compaction	Leveled Compaction
* Done by Cassandra	* Done by RocksDB, HBase
* When enough SSTables of similar size are accumulated, then compaction process is trigger. smaller SST merged to Larger SSTs.	* Level 0 → (SSTables) → some size. when size of merged SSTable grows, it is promoted to next level. (Level 0 → Level 1)
	* Pages within same level go through compaction.

LSM tables, because writes are sequential. Hence very high write throughput.

(B-Trees)

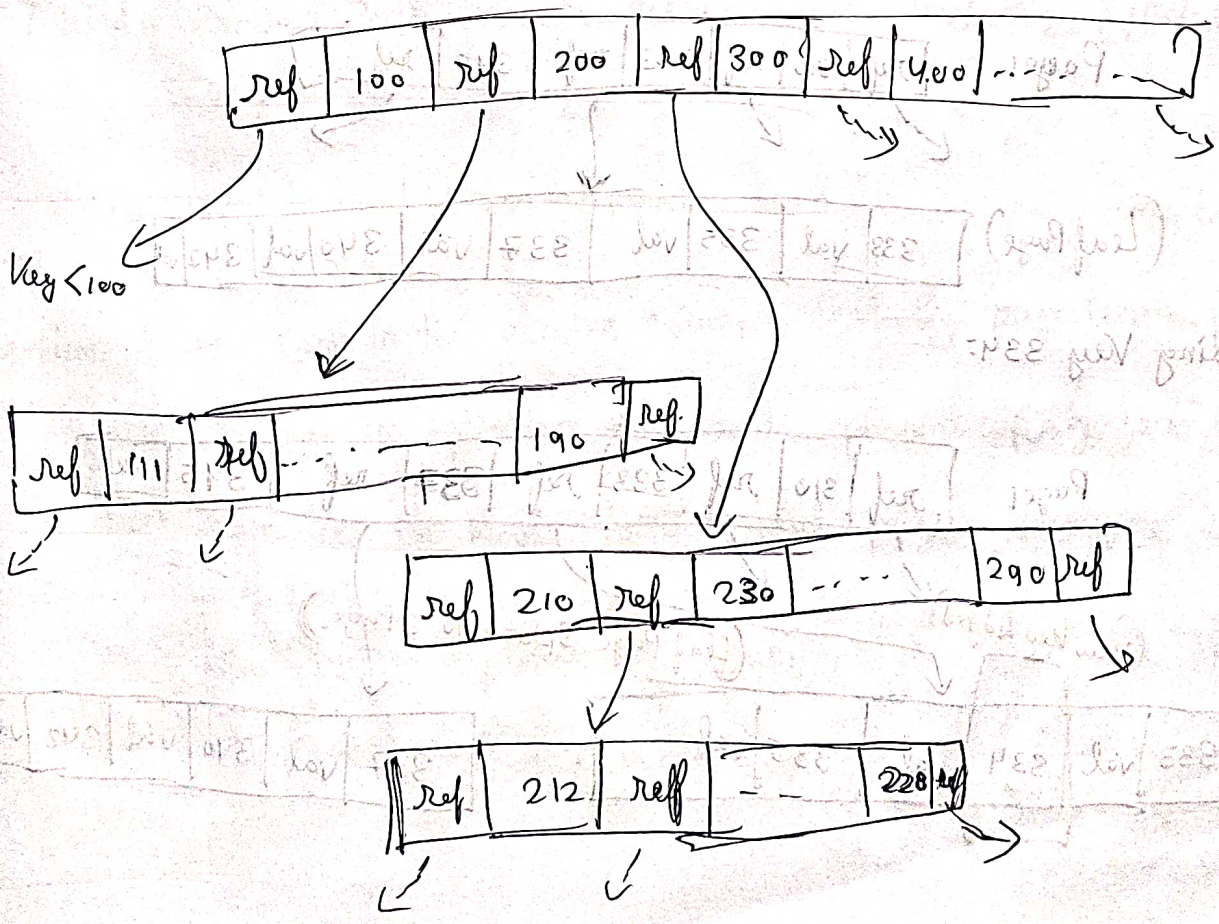
Most widely used indexing structure,

B-trees have a very different design philosophy

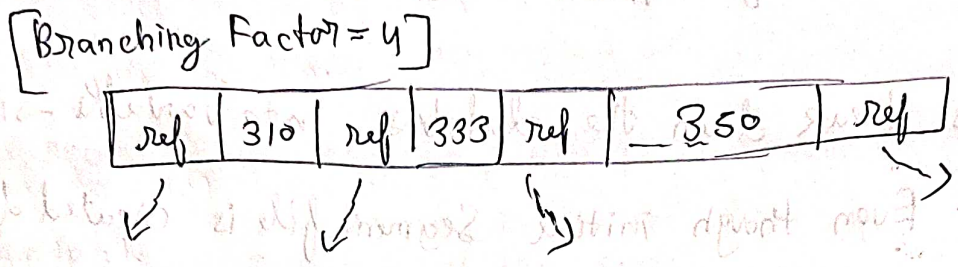
LSM trees break down the database into variable-size segments. Even though initial segment file is created of a fixed size, But upon merging size becomes variable.

B-trees by contrast breaks the database down to fixed-size blocks or pages, traditionally 4KB in size.

Design is some what like segment tree, Except it has multiple children.



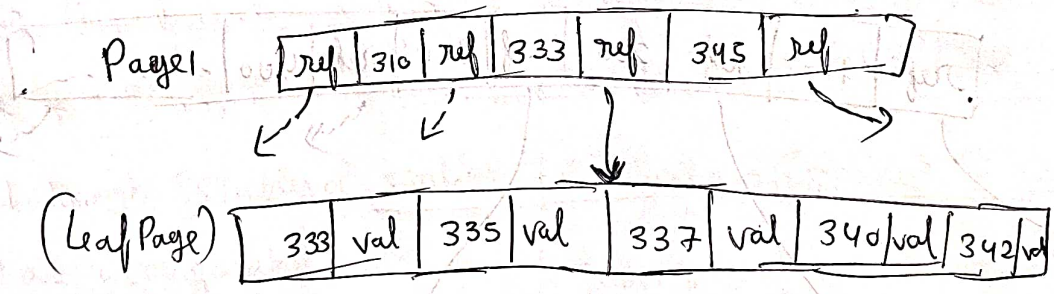
The number of references to child pages in one page of B-tree is called Branching factor.



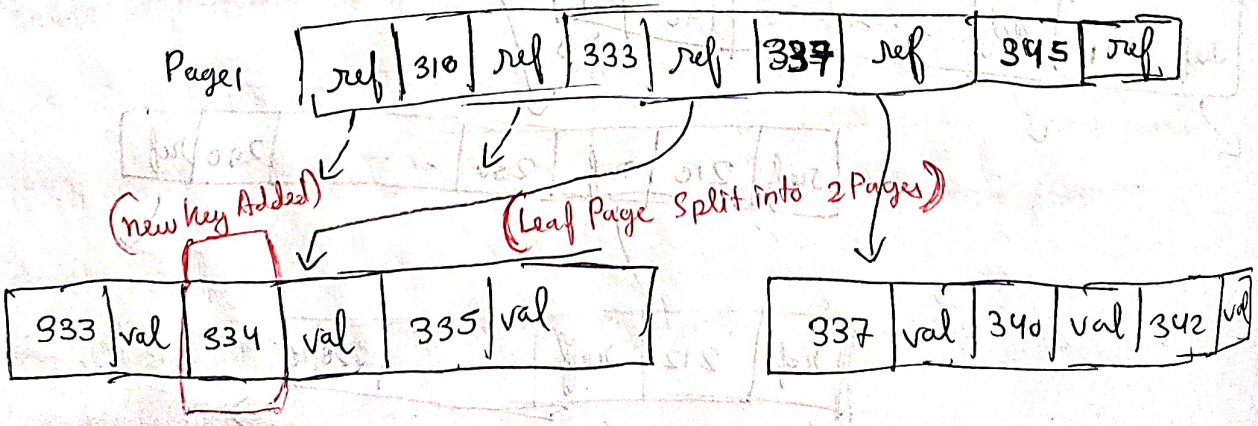
Update Operation → $O(\log_K N)$ (N → number of keys) K → Branching Factor

If you want to update the value for an existing key in a B-tree, you search for the leaf page containing the key, and it has the offset to the file. Value is edited.

(Lets take example of a Key Addition)



Adding key 334:



Most Databases can fit into B-tree that is (4-3) layers deep, we don't need to lookup deep.

A B-tree of 4KB Page with Branching factor of 500 stores (256TB)

(Making B-trees reliable)

- Basic underlying write operation of a B-tree is to overwrite a page on disk with new data. It is assumed overwrite does not change the location of the page i.e. all references to that page remain intact when the page is overwritten.

Whereas LSM trees only append to files but never modify the files in place.

(Kind of Random write, except the magnetic head can find the key easily).

Btree → Modify LSM tree - Append.

Problem: As we saw in the previous example, that if page becomes full, the ~~parent~~ page is split into two and parent page is modified are references are made to both child pages. Imagine if during making references the server crashes and one child page is lost. How to handle it?

A) Inorder to make the database resilient, we maintain an additional data-structure called WAL (Write Ahead Log).

↳ It stores the modification that was suppossed to be made before doing the operation.

When the server comes back up, the Log is read and operation is completed.

[Concurrency]

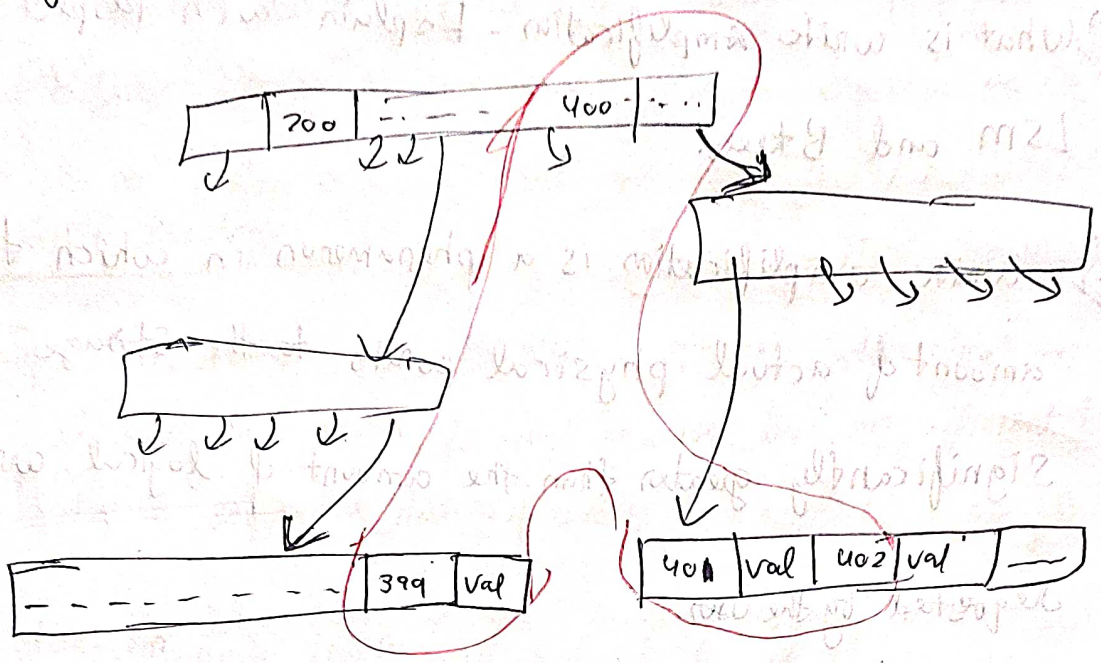
Since in B-trees the actual address/location is modified. So we need to apply thread locks (latches) properly.

LSM trees are simpler in this regard. The writes can happen in the another thread and later in background threads this merging and compaction is taken care and old data is overriden by new data

(B tree optimisations)

* If we abbreviate keys we can store more keys in a single page. Hence more branching factor. Hence lesser no of levels and faster search.
* Additional left and right pointers have been added to the tree. Each leaf page has additional pointers to left and right sibling leaf pages. So that we don't have to jump on parent page again.

* Lets say B tree structure is somewhat like.



Now you do a range query on (399 - 402)
You see how much disk seek time is required for every page read. It might have been. we might have to read 3 leaf pages in different subtrees.

LSM trees make range queries easier in this sense and because they are Sorted String Tables and with compaction, they get the ~~correct~~ fast. result

* Copy-on-write (COW) is a strategy where modifications are not made on the data completely. The data/page which is to be modified. ~~First its copy is created.~~

1). 1st copy of Page is created
2) Modifications are done on the copy
3). Modifications after verified. The references are made to point to new pages.

{ this helps in Repeatable Reads concepts, where the transaction reads only ~~reads~~ commits before the transaction begins }

Q) What is write Amplification. Explain with Respect to LSM and B trees.

A) Write amplification is a phenomenon in which the amount of actual physical writes to the storage is significantly greater than the amount of logical writes requested by the user.

(Write Amplification w.r.t B-Trees)

Insertion in B-Tree:

Suppose we have a B-Tree where each node can hold upto 3 keys. If we insert keys in this page, it will cause huge write amplification as it will lead to propagating the splits to root of the tree

Write Amplification w.r.t LSM trees (Compaction):

Since during compaction the two SSTables are merged and compacted and new SSTable is created and written to disk. Lot of new keys are rewritten, So High write amplification.

Q) Why in general LSM trees have lower write amplification than B trees?

A) Compaction Strategy:

LSM tree: Compaction merges multiple SSTables into single one, Although that involves frequent re writes but are efficient due to sequential nature of writes

B-trees: Frequent node splits → lead to propagating to whole tree increase write amplification very high.

Rebalancing

LSM tree: There is no overhead of Rebalancing just periodic compaction.

B-Tree Require Rebalancing to maintain tree structure, which involves additional writes.

Log Structured Nature.

LSM Tree: Since these trees follow a sequential write and log structured approach. Space is utilised very nicely and minimises fragmentation.

B-Trees: Random writes are done here and there and in order to main sorted order, and Rebalancing some pages have unused memory [higher fragmentation]

(* * *)

In the market, on many SSDs are internally using the log-structured algorithm to turn random writes to sequential writes. [Faster SSDs]

(* * *)

If any storage engine has high write-amplification, So it can perform fewer writes per second, given constant available disk bandwidth is there

(Downsides of LSM-trees)

1). Even though storage engines try to perform compaction incrementally and without affecting concurrent disk ~~access~~ access, ~~limited~~ It can so easily happens that compaction thread eats up the CPU and does not give the user access thread a chance

2). Lets say a database is completely new.

Two operations

1. Memtable getting closer to full → Flushed to SSTable
2. SSTables getting merged and new SSTable is created.

Both operations require disk bandwidth,

So proper monitoring is required such that compaction also gets a chance, memtable flushing and user thread gets a chance [Complex System]

Advantage of B-tree is that each key exists exactly in one place in the index whereas LSM tree has multiple copies of same key.

This aspect of B-trees makes it very strong candidate for transactional databases. It lets us apply locks on range of keys.

B-trees have provided consistently good performance for many workloads. Similarly log-structured ~~trees~~ merge trees are gaining popularity as well

LSM Trees	B Trees.
1) Faster of writes	1) Faster for reads
2) Sustain high write throughput due to less amplification of write.	2) Write overhead high due to high write amplification
3) Can be compressed better. Compaction takes care of reducing fragmentation	3) Leaves some space unused Causes fragmentation.
4) Compaction process can affect ongoing read/write → Observed on high write/read throughput	4) Each key is present in a single place, Good support for transactional requirements

(Other Indexing Options)

Keep a secondary index, two maybe find results faster based on query patterns

(Storing values within the index)

Key on which the query is done on, is actually searched in the B-tree, the value it stores can be two things.

1. Actual row
2. Reference to the location in file
   ↳ In this case where the data is stored is called heap file.

   Heap file approach is common because it avoids duplicating data when multiple secondary indexes are present. Each index just refers to the location in the heap file

In case of overwriting if the new value that is supposed to be written is larger than old value holed location. Then new value needs to be written in new location and reference has to be changed.

In some situations, the extra hop from index to heap file is costly for reads, so it is desirable to keep the whole row in the index :→ Clustered index.

(**) A compromise between a clustered index (storing all columns of a row) and non-clustered index (storing references) is known as covering index or index with included columns

Only those columns which are used by user frequently

As with any kind of data duplication, clustered and covering indexes can speed up reads. But require additional storage and overhead on writes.

Multi-Column Indexes

Indexing on two or more columns together.

Example (lastName, firstName)

Ex: When user is looking at restaurants on a map, the website, we wants to search within a range.

Select * from restaurant WHERE lat > x AND lat < y
AND long > w AND long < v.

(LSM, B trees) ⇒ cannot index both columns simultaneously.

R-tree is one such implementation, such that indexing multiple dimensional information such as geographical coordinates.

(For example) Index on (Red, blue, green) for choosing clothes in range for this (R_1, G_1, B_1) to $(R_2 G_2 B_2)$

(Full text search Fuzzy Index)

Fuzzy index → Basically Lucene provides a facility where we can search for words with (edit Distance = 1). Any 1 letter is not matching we will display that as well.

(Keep Everything In Memory)

Memcached → (K-V) store for caching.

Recovery → write log in disk and RAM is backup by Battery.

Cache Eviction Policies are there → (LRU, LFU)

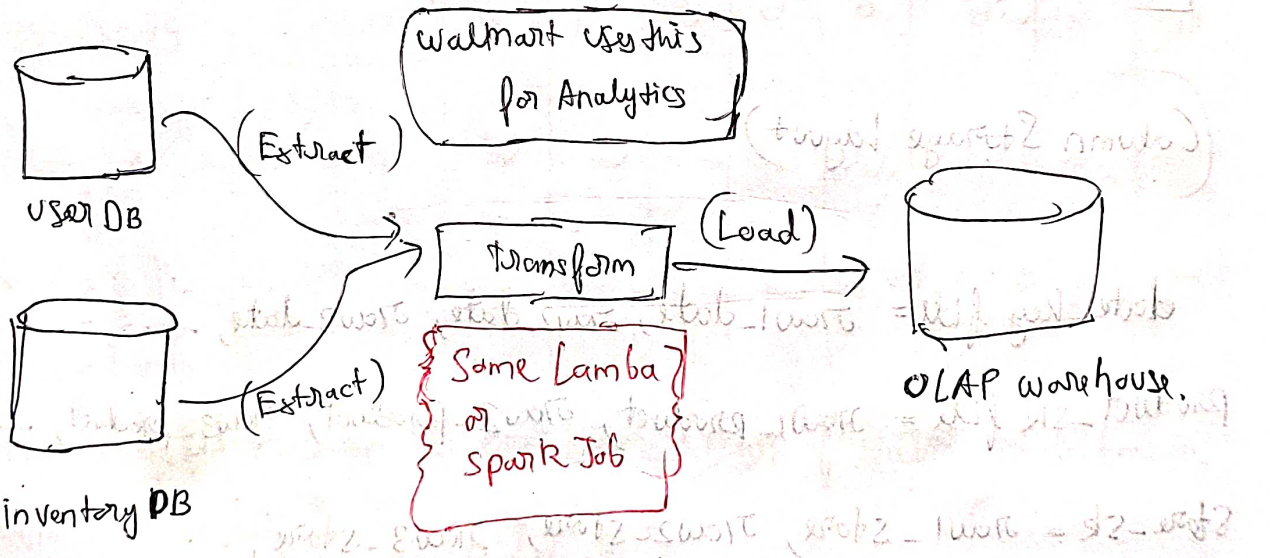
(Redis - Couchbase) provide weak durability as they are writing log to disk asynchronously.

In-memory data structure much easier to maintain.

Its basically priority-queue or Hashset versus LSM trees

Use Case → Transaction Processing or Analytics.

OLTP (Transactional) (Processing)	OLAP (Analytic) (Processing)
1) Read small number of records per query fetched by key	1) Aggregate over large set of data
2) write - Random Access, like B trees. (low latency write)	2) ETL (Extract Transform Load)
3) Medium Size	3) Very Big Size Data



~~Column~~

(Column Oriented Storage)

Storing columns row ~~wise~~ wise makes search and query fast sometimes.

Example:

Table

date_key	product_sk	store_sk	promotion_sk	— — —

(Column Storage Layout)

date_key file = row1_date, row2_date, row3_date, — — —

product_sk file = row1_product, row2_Product, row3_product, —

Store_sk = row1_store, row2_store, row3_store, — — —

promotion_sk file = row1_promo, row2_promo, — — —

Advantage of storing them in this format is, we can only fetch out those columns which are required and we don't need entire table.

Column Compression.

Bitmap Encoding

Column.

product_sk:	69	69	69	74	31	31	31	29	30	31
Bitmap for each possible value.										
product_sk = 29	0	0	0	0	0	0	0	1	0	0
product_sk = 69	1	1	1	0	0	0	0	0	0	0
Product_sk = 31	0	0	0	0	1	1	1	0	0	1
⋮										
Product_sk = 74	0	0	0	1	0	0	0	0	0	0

These can be generated into bitmasks and in column we can have a map of value to bitmasks (Provided n is small).

Product_sk: (29 → 189132, 69 → 200100, 31 → 1002) ⇒ Bitmasks.

Vectorised Processing (name of technique)

If query has WHERE product_sk IN (29, 69, 31).

We can just take the bitmap and do bit_{29} OR bit_{69} OR bit_{31}

For where product_sk = 31 AND store_sk = 3, just do

(Bitwise AND to get both common rows.)